WiSH Digital HW — Pre-Work Instructions for Mentees

Overview

Welcome to the **WiSH** program! To make the most of your classroom sessions, please complete the following pre-work **before Day 1**. The pre-work is organized by track/day and includes video content, reading material, and tool installation instructions.

📅 Schedule Reference

Date	Day	Primary Topics
5/25	Monday	Intro to Semiconductor Design + Architecture & RTL
5/26	Tuesday	Architecture & RTL (continued) + Design Verification (DV)
5/27	Wednesday	Design Verification (DV)
5/28	Thursday	DFT – Design for Test
5/29	Friday	Physical Design (PD)
6/12	Friday	MSP Workshop
6/15–6/16	Mon–Tue	MSP Workshop Labs (continued)

🗒️ Week1 — Foundations Pre-Work

📖 What to Expect

Before diving into the hands-on sessions, Day 1 lays the foundation for everything you will encounter throughout the WiSH program. You will start by understanding what a VLSI chip is — from the smallest building block (a transistor) all the way up to a complete chip. Along the way, you'll explore number systems (binary, hexadecimal, and more), Boolean algebra, and techniques like K-Maps used to simplify logic expressions. By the end of day, you should have a solid grasp of how digital hardware thinks and computes — setting you up for success in RTL coding, verification, physical design, and beyond.

📺 Video Content

- 🎦 An Introduction to Semiconductor Device lifecycle

🗒️ Week1 — Architecture & RTL Pre-Work

System architecture forms the foundation of modern digital design. This overview introduces the key concepts and building blocks you'll encounter. Think of system architecture as the **"blueprint"** that defines how different components of a **digital system interact and communicate** with each other.

Please refer following document for more information on System Architecture.

📺 Video Content

Verilog Basics

- 🎦 An Introduction to Verilog
- 🎦 VERILOG LANGUAGE FEATURES (PART 1)
- 🎦 VERILOG LANGUAGE FEATURES (PART 2)
- 🎦 VERILOG LANGUAGE FEATURES (PART 3)

📝 Exercises (Complete before Day 1)

1. Write a simple **AND gate** module in Verilog
2. Create a **2-to-1 Multiplexer** in Verilog
3. Explain the functionality of the following code block:

```
module decoder_2to4_enable_n (
   input [1:0] sel,
   input enable_n,
   output [3:0] y
);
   wire en = ~enable_n;
   assign y[0] = en & ~sel[1] & ~sel[0];
   assign y[1] = en & ~sel[1] &  sel[0];
   assign y[2] = en &  sel[1] & ~sel[0];
   assign y[3] = en &  sel[1] &  sel[0];
endmodule
```

🛠️ Tool Installation — iVerilog & GTKWave

⚠️ **All mentees must install iVerilog and GTKWave before Day 1 classroom session.**

Windows Users:

1. Download the latest version from: https://bleyer.org/icarus/
2. Open Windows Settings (Win + I) → Search for **"Advanced App Settings"**
3. Under *"Choose where to get apps"*, select **"Anywhere"**
4. Run the downloaded installer
5. If you see *"Windows protected your PC"*, click **More Info → Run Anyway**
6. Select all **default options** during installation

📝 **Note:** On Windows, the latest iVerilog already includes GTKWave — no separate installation needed.

Mac Users:

Follow the video guide: **[iVerilog + GTKWave Installation Guide for Mac]**

✅ **Validation:**

After installation, open a terminal and run:
iverilog -v
vvp -v
You should see version information printed. If not, revisit the installation steps.

🗓️ Week1 — Design Verification (DV) Pre-Work

📖 What is Design Verification?

Verification of the functional behavior of an RTL file against device specifications. The goal is exhaustive verification to cover all features of the design.
⚠️ **Complete the Architecture & RTL Pre-Work before starting DV Pre-Work.**

📺 Verilog Concepts to Review

Study the following Verilog topics (use online resources or provided materials):

- Verilog initial Block
- Verilog Testbench
- Verilog Display Tasks
- Verilog $random
- Verilog Task
- Verilog Functions

📝 These concepts are essential for building effective **testbenches** to verify a Design Under Test (DUT).

🛠️ Tool Validation — iVerilog & GTKWave

Ensure iVerilog and GTKWave (installed in Day 1 prep) are working correctly by simulating a basic testbench before the DV session.

🗓️ Week1 — DFT – Design for Test Pre-Work

📖 What is DFT?

Add **controllability and observability** to the design for structural testing. DFT validates that components work as per their function (structural correctness — not necessarily functional behavior from specifications).

📺 Video Content

DFT Basics

- 🎦 An Introduction to Design For Testability

📝 Key Concepts to Review

- Why controllability and observability matter in silicon testing

---

🗓️ Week1 — Physical Design (PD) Pre-Work

📺 Video Content

VIDEO: **WISH26_PD_final_Pre_work** *Covers the full Physical Design flow including partitioning, floor planning, placement, routing, and verification.*

📖 What is Physical Design?

Transform a circuit's **logical design into a physical layout** ready for fabrication. Steps include:

- Partitioning
- Floor planning
- Placement
- Routing
- Verification

Goals: Optimal performance, low power consumption, and minimal area (PPA).

📺 Supporting Videos to Watch

1. Semiconductor Manufacturing Process

🛠️ Tool Installation — OpenLane

⚠️ Install OpenLane before Day 5. Follow the correct guide for your OS.

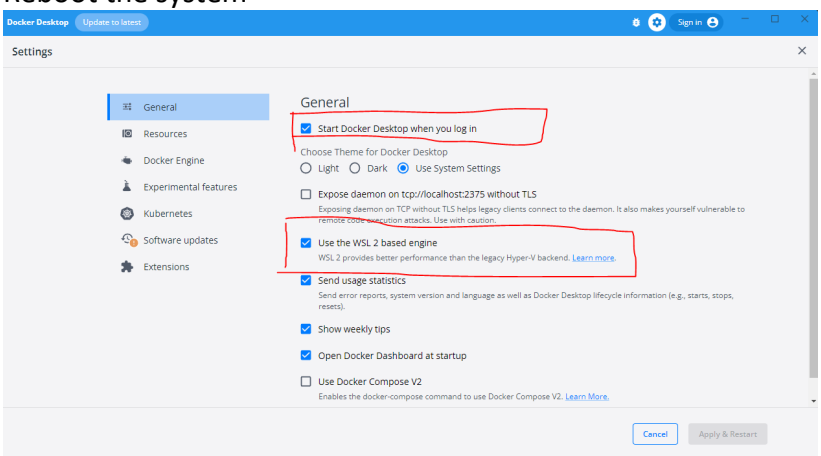
Windows Users:

Step 1 : Install wsl

- The Windows Subsystem for Linux (WSL) lets developers install a Linux distribution
- Steps to install
  - Run windows powershell as administrator
  - Run wsl –install
  - Reboot the system
  - Open wsl and set password
- Installing WSL on Windows has instructions to install wsl (also has commonly used linux commands)

Step 2 : Install Docker

- https://docs.docker.com/desktop/setup/install/windows-install/
- Steps to install
  - Download and install docker from website mentioned above
  - Open Docker and make sure settings marked in image below are enabled
  - Reboot the system



Step 3 : Install OpenLane

Steps to install

- Open wsl
- Run following commands
  1. mkdir Lab
  2. cd Lab

3. sudo apt-get update
4. sudo apt-get upgrade
5. sudo apt install -y build-essential python3 python3-venv python3-pip make git
6. docker run hello-world
7. git clone --depth 1 https://github.com/The-OpenROAD-Project/OpenLane.git
8. cd OpenLane/
9. make
10. make test

✅ A successful installation will output: **Basic test passed**

📎 Full reference: OpenLane Windows Installation Guide

Mac Users:

📎 Full reference: OpenLane Mac Installation Guide

Installation of required packages

First install Homebrew then run script below to install the required packages:

Mandatory to install homebrew before going to the next steps

```
brew install python make
brew install --cask docker
```

If the above does not work, try the below commands and try again.

```
echo >> /Users/<username>/.zprofile
echo 'eval "$(/opt/homebrew/bin/brew shellenv zsh)"' >> /Users/<username>/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv zsh)"
```

Checking the Docker Installation

After that, you can run Docker Hello World without root. To test it use the following command:

```
# After reboot
docker run hello-world
```

You will get a little happy message of Hello world, once again, but this time without root.

```
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
```

```
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Troubleshooting docker installation issues [Linux/Ubuntu only]

If you get Docker permission error when running any Docker images, then likely, you forgot to follow the steps to make Docker available without root or you need to *restart your Operating System*.

```
OpenLane> docker run hello-world
docker: Got permission denied while trying to connect to the Docker daemon socket at
unix:///var/run/docker.sock: Post
"http://%2Fvar%2Frun%2Fdocker.sock/v1.24/containers/create": dial unix
/var/run/docker.sock: connect: permission denied.
See 'docker run --help'.
OpenLane>
```

Checking Installation Requirements

In order to check the installation, you can use the following commands:

```
git --version
docker --version
python3 --version
python3 -m pip --version
make --version
python3 -m venv -h
```

Successful output will look like this:

```
git --version
docker --version
python3 --version
```

```
python3 -m pip --version
make --version
python3 -m venv -h
git version 2.36.1
Docker version 20.10.16, build aa7e414fdc
Python 3.10.5
pip 21.0 from /usr/lib/python3.10/site-packages/pip (python 3.10)
GNU Make 4.3
Built for x86_64-pc-linux-gnu
Copyright (C) 1988-2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
usage: venv [-h] [--system-site-packages] [--symlinks | --copies] [--clear]
            [--upgrade] [--without-pip] [--prompt PROMPT] [--upgrade-deps]
            ENV_DIR [ENV_DIR ...]

Creates virtual Python environments in one or more target directories.
...
Once an environment has been created, you may wish to activate it, e.g. by
sourcing an activate script in its bin directory.
```

Download and Install OpenLane

Download OpenLane from GitHub:

```
git clone --depth 1 https://github.com/The-OpenROAD-Project/OpenLane.git
cd OpenLane/
make
make test
```

These steps will download and build OpenLane and sky130 PDK. Finally, it will run a ~5 minute test that verifies that the flow and the PDK were properly installed. If you are planning to use another PDK, then you need to follow the PDK installation guide for that specific PDK.

Successful test will output the following line:

```
Basic test passed
```

Search for amd64 in all files and replace with arm64 for mac.

Optional: Viewing Test Design Outputs

Open the final layout using KLayout. This will open the window of KLayout in editing mode `-e` with sky130 technology.

```
# Enter a Docker session:
make mount

# Open the spm.gds using KLayout with sky130 PDK
klayout -e -nn $PDK_ROOT/sky130A/libs.tech/klayout/tech/sky130A.lyt \
  -l $PDK_ROOT/sky130A/libs.tech/klayout/tech/sky130A.lyp \
  ./designs/spm/runs/openlane_test/results/final/gds/spm.gds

# Leave the Docker
exit
```

Note: Use Google AI for any other installation issues. The instructions in the manual may be outdated and might have to be replaced or other environment variables and src files might have to be created for the tool to work.

📝 Key Concepts to Review (PD)

- CMOS Working Principle (NMOS + PMOS as switches)
- CMOS Inverter, NAND, NOR gate implementations
- Technology Nodes (nm-scale features)
- Power Dissipation: Leakage Power, Switching Power, Internal Power
- **Static Timing Analysis (STA):**
  - Timing paths, tcq, tsetup, thold
  - Setup (MAX) constraints — sets maximum clock frequency
  - Hold (MIN) constraints — independent of clock period; hold failures are fatal!
  - False paths

🗒️ Week3/4 (6/12, 6/15–6/16) — MSP Workshop Pre-Work

📖 What is the MSP Workshop?

The MSP Workshop introduces you to **embedded systems programming** using TI's **MSPM0G3507 microcontroller**. You will write C code to control hardware peripherals, understand microcontroller architecture, and build small embedded applications.

📚 Step-by-Step Pre-Work Instructions

⚠️ **Follow the order below strictly. Do not skip ahead.**

Step 1 — Learn Embedded Software Fundamentals (Start Here)

Primary Resource:

🎓 Introduction to Embedded Systems using MSPM0+ — Dr. Valvano, UT Austin

- Provides in-depth introduction to **C-programming** & **Embedded SW** using MSPM0
- **Go through all lecture slides** from this link before moving to Step 2

Supplementary Resource:
📘 Embedded Programming for Cortex-M Devices

- Good reference for general embedded hardware and Cortex-M programming concepts

---

Step 2 — Get Familiar with the MSPM0G3507 (After Step 1)

Review the following TI resources for the target microcontroller:

Resource	Link
MSPM0G3507 Product Page	ti.com/product/MSPM0G3507
MSPM0G3507 Datasheet	ti.com/lit/gpn/mspm0g3507
MSPM0G3507 Technical Reference Manual	ti.com/lit/pdf/slau846

Step 3 — Watch MSPM0 Peripheral Video Series (After Step 2)

📺 Video Series on MSPM0 Peripherals

Watch the video series to understand how individual peripherals (GPIO, UART, SPI, I2C, ADC, Timers, etc.) work on the MSPM0 family.

📝 Key Concepts to Be Familiar With for the Workshop

Topic	Why It Matters
Basic C programming (variables, loops, functions, pointers)	All embedded code is written in C
Digital I/O (GPIO)	Controlling LEDs, reading buttons
Microcontroller memory map	Understanding how peripherals are memory-mapped
Clock systems	How the MCU generates and uses clock signals
Interrupts	Responding to hardware events asynchronously
Communication protocols (UART, SPI, I2C)	Interfacing with sensors and other devices

💡 **Tip:** If you run into any installation issues, reach out to your assigned mentor before the classroom session. You/assigned mentor can reach out to Pervez Garg (pervezgarg@ti.com) for any further help.